

Network Automation and Integrated Security Challenge

Laboratory 5

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	6 to 8 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name __Haruna Muhammad Osori

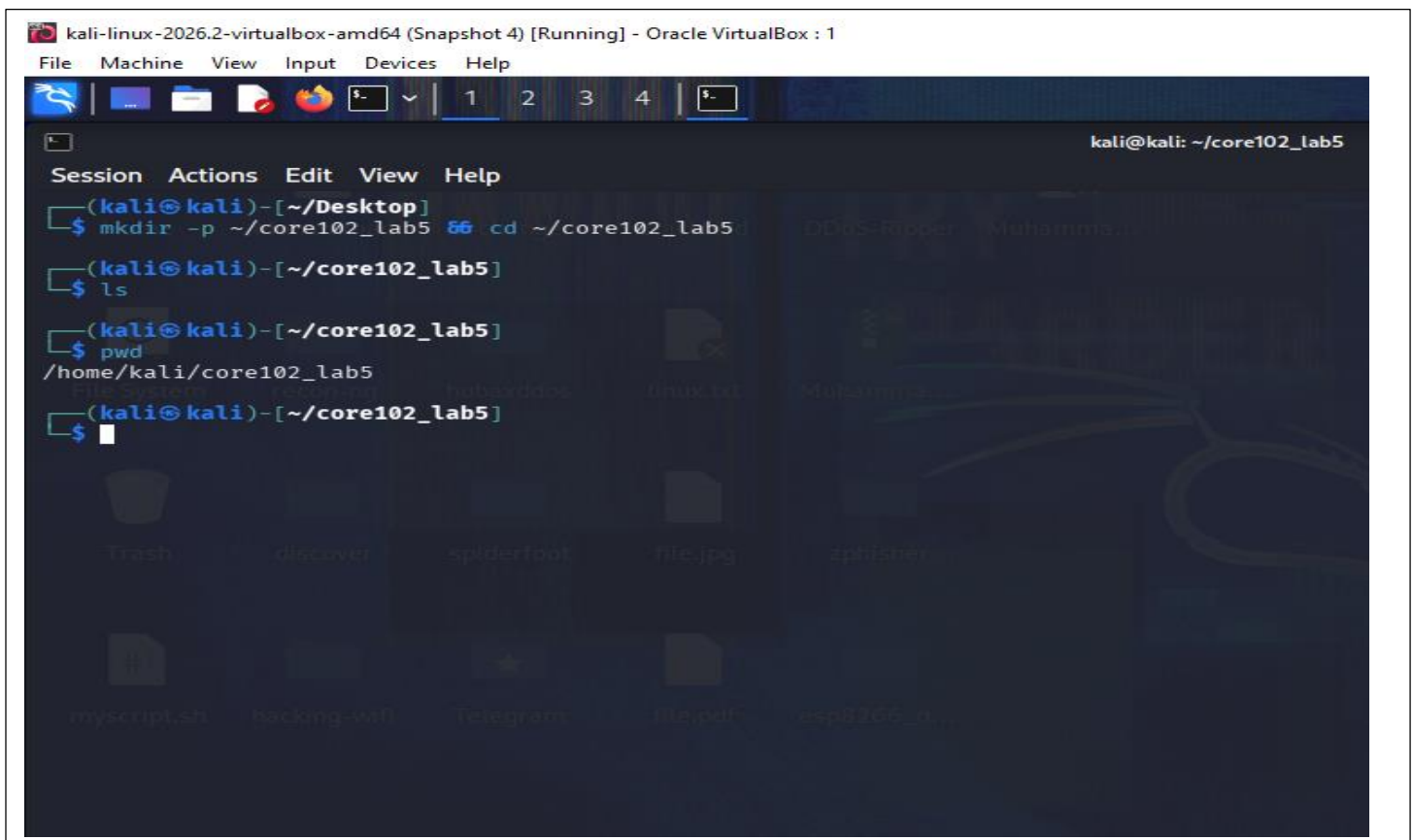
Registration number _____

Submission date _____

Part 1 Automation Plan and Scope

Before writing any code, I set the rules for each tool. The inventory script only reads information and changes nothing. The ping sweep is limited to the 192.168.56 network with a one-second timeout. The service checker only accepts addresses in 192.168.56.0/24 and a fixed list of ports (22, 53, 80, 443 and 8080). The PCAP script only reads a saved file and never captures live traffic. These limits keep every tool inside the authorised lab.

Tool	Permitted input	Output	Timeout or limit	Failure behaviour
Network inventory	Local host only	Timestamped text	Read-only commands	Report command errors
Ping sweep	192.168.56.1 to .254	Responsive hosts	One second per probe	Reject other prefix
Service checker	One authorised host	Selected port state	One second per port	Reject invalid host
PCAP summary	Existing local PCAP	Protocol counts	Maximum file supplied	Reject missing file



```
kali@kali: ~/core102_lab5
$ mkdir -p ~/core102_lab5
$ cd ~/core102_lab5
$ ls
$ pwd
/home/kali/core102_lab5
```

Part 2 Bash Network Inventory

I wrote `network_inventory.sh` to record the interfaces, routes, neighbours, listening sockets, DNS settings and disk space into a timestamped report. It uses strict mode (`set -euo pipefail`) so it stops on errors and undefined variables. I ran it with the default folder and again with a custom one, and both created a report. I added a section showing the default gateway and the route to 192.168.56.1. The script only uses read-only commands, so it doesn't change the system.

```
#!/usr/bin/env bash
set -euo pipefail

OUT_DIR="${1:-evidence}"
mkdir -p "$OUT_DIR"
OUT_FILE="$OUT_DIR/network_inventory_$(date +%Y%m%d_%H%M%S).txt"

section() {
    echo
    echo "===== $1 ====="
}

{
    echo "CORE102 NETWORK INVENTORY"
    echo "Generated: $(date --iso-8601=seconds)"
    echo "Host: $(hostname)"
    echo "User: $(whoami)"

    section "INTERFACES"
    ip -br link
    ip -br addr

    section "ROUTES"
    ip route

    section "NEIGHBOURS"
    ip neigh show

    section "LISTENING SOCKETS"
    ss -tuln

    section "RESOLVER"
    cat /etc/resolv.conf

    section "DISK SPACE"
    df -h .
} > "$OUT_FILE"

echo "Inventory saved to $OUT_FILE"

chmod +x network_inventory.sh
bash -n network_inventory.sh
./network_inventory.sh
./network_inventory.sh custom_evidence
find . -maxdepth 2 -type f -name 'network_inventory *.txt' -print
sed -n '1,160p' evidence/network_inventory_*.txt
```

```
kali@kali: ~/core102_lab5

Session Actions Edit View Help

(kali@kali)~[~/Desktop]
$ mkdir -p ~/core102_lab5 66 cd ~/core102_lab5

(kali@kali)~[~/core102_lab5]
$ ls

(kali@kali)~[~/core102_lab5]
$ pwd
/home/kali/core102_lab5

(kali@kali)~[~/core102_lab5]
$ nano network_inventory.sh

(kali@kali)~[~/core102_lab5]
$ ls
network_inventory.sh

(kali@kali)~[~/core102_lab5]
$ chmod +x network_inventory.sh

(kali@kali)~[~/core102_lab5]
$ bash -n network_inventory.sh

(kali@kali)~[~/core102_lab5]
$ sudo ./network_inventory.sh
[sudo] password for kali:
Inventory saved to evidence/network_inventory_20261003_045249.txt

(kali@kali)~[~/core102_lab5]
$ ./network_inventory.sh custom_evidence

(kali@kali)~[~/core102_lab5]
$ ls
custom_evidence  evidence  network_inventory.sh

(kali@kali)~[~/core102_lab5]
$ find . -maxdepth 2 -type f -name 'network_inventory_*.txt' -print
./custom_evidence/network_inventory_20261003_045337.txt
./evidence/network_inventory_20261003_045249.txt

(kali@kali)~[~/core102_lab5]
$ sed -n '1,160p' evidence/network_inventory_*.txt
CORE102 NETWORK INVENTORY
Generated: 2026-10-03T04:52:49-04:00
Host: kali
User: root

===== INTERFACES =====
lo          UNKNOWN    00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0        UP         08:00:27:5a:87:bc <BROADCAST,MULTICAST,UP,LOWER_UP
>
eth1        UP         08:00:27:0a:6f:b3 <BROADCAST,MULTICAST,UP,LOWER_UP
>
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP         10.80.67.204/24 fe80::fc2a:17d4:b059:edc7/64
eth1        UP         192.168.56.101/24 fe80::d9bd:df5b:c2a0:f2bd/64

===== ROUTES =====
default via 10.80.67.147 dev eth0 proto dhcp src 10.80.67.204 metric 100
10.80.67.0/24 dev eth0 proto kernel scope link src 10.80.67.204 metric 100
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 101
```

```
kali@kali: ~/co
Session Actions Edit View Help
(kali@kali)~[~/core102_lab5]
$ sed -n '1,160p' evidence/network_inventory_*.txt DDoS Report - Munirina
CORE102 NETWORK INVENTORY
Generated: 2026-10-03T04:52:49-04:00
Host: kali
User: root

===== INTERFACES =====
lo UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0 UP 08:00:27:5a:87:bc <BROADCAST,MULTICAST,UP,LOWER_UP>
eth1 UP 08:00:27:0a:6f:b3 <BROADCAST,MULTICAST,UP,LOWER_UP>
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 10.80.67.204/24 fe80::fc2a:17d4:b059:edc7/64
eth1 UP 192.168.56.101/24 fe80::d9bd:df5b:c2a0:f2bd/64

===== ROUTES =====
default via 10.80.67.147 dev eth0 proto dhcp src 10.80.67.204 metric 100
10.80.67.0/24 dev eth0 proto kernel scope link src 10.80.67.204 metric 100
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 101

===== NEIGHBOURS =====
192.168.56.100 dev eth1 lladdr 08:00:27:71:c1:58 STALE

===== LISTENING SOCKETS =====
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5355 0.0.0.0:*
udp UNCONN 0 0 [::]:5355 [::]:*
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
tcp LISTEN 0 4096 0.0.0.0:5355 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
tcp LISTEN 0 128 [::]:22 [::]:*
```

```
===== RESOLVER =====
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .

===== DISK SPACE =====
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        79G   20G   55G   27% /
```

Part 3 Controlled Bash Ping Sweep

I wrote `authorised_ping_sweep.sh` to check hosts from 192.168.56.1 to .254 with one ping each and a one-second timeout. I tested the scope control first by running it against 203.0.113. It was rejected with exit status 2 before any ping was sent. The real sweep found 192.168.56.100, .101 and .102. The .101 address is my Kali client and .102 is the server. The host adapter at .1 did not answer, because a host can be online and still ignore ping. I improved the script to record the start time, finish time and the number of hosts that responded.

```
#!/usr/bin/env bash
set -u

PREFIX="${1:-192.168.56}"
OUT="live_hosts_$(date +%Y%m%d_%H%M%S).txt"

if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
    exit 2
```

```

fi

echo "Authorised ICMP sweep of ${PREFIX}.0/24" | tee "$OUT"
for host in $(seq 1 254); do
    address="${PREFIX}.${host}"
    if ping -c 1 -W 1 "$address" >/dev/null 2>&1; then
        echo "$address responded" | tee -a "$OUT"
    fi
done
echo "Saved: $OUT"

chmod +x authorised_ping_sweep.sh
bash -n authorised_ping_sweep.sh
./authorised_ping_sweep.sh 203.0.113
echo "Rejected exit status: $?"
./authorised_ping_sweep.sh 192.168.56
cat live_hosts_*.txt

```

```

kali@kali: ~/core102_lab5
Session Actions Edit View Help
(kali@kali)-[~/core102_lab5]
$ ls
custom_evidence  evidence  network_inventory.sh
(kali@kali)-[~/core102_lab5]
$ nano authorised_ping_sweep.sh
(kali@kali)-[~/core102_lab5]
$ chmod +x authorised_ping_sweep.sh
(kali@kali)-[~/core102_lab5]
$ bash -n authorised_ping_sweep.sh
(kali@kali)-[~/core102_lab5]
$ ./authorised_ping_sweep.sh 203.0.113
Scope rejected. Authorised prefix: 192.168.56
(kali@kali)-[~/core102_lab5]
$ sudo ./authorised_ping_sweep.sh 203.0.113
Scope rejected. Authorised prefix: 192.168.56
(kali@kali)-[~/core102_lab5]
$ echo "Rejected exit status: $?"
Rejected exit status: 2
(kali@kali)-[~/core102_lab5]
$ sudo ./authorised_ping_sweep.sh 192.168.56
Authorised ICMP sweep of 192.168.56.0/24

```

Part 4 Python Service Checker

I wrote `service_check.py` to test TCP ports on one host with a one-second timeout. I tested it with bad input: `abc` was rejected as an invalid address

```

#!/usr/bin/env python3
import ipaddress
import socket
import sys
from datetime import datetime

AUTHORISED = ipaddress.ip_network('192.168.56.0/24')
PORTS = [22, 53, 80, 443, 8080]

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} HOST")
    raise SystemExit(1)

try:
    target = ipaddress.ip_address(sys.argv[1])
except ValueError:
    print("Invalid IPv4 address")

```

```

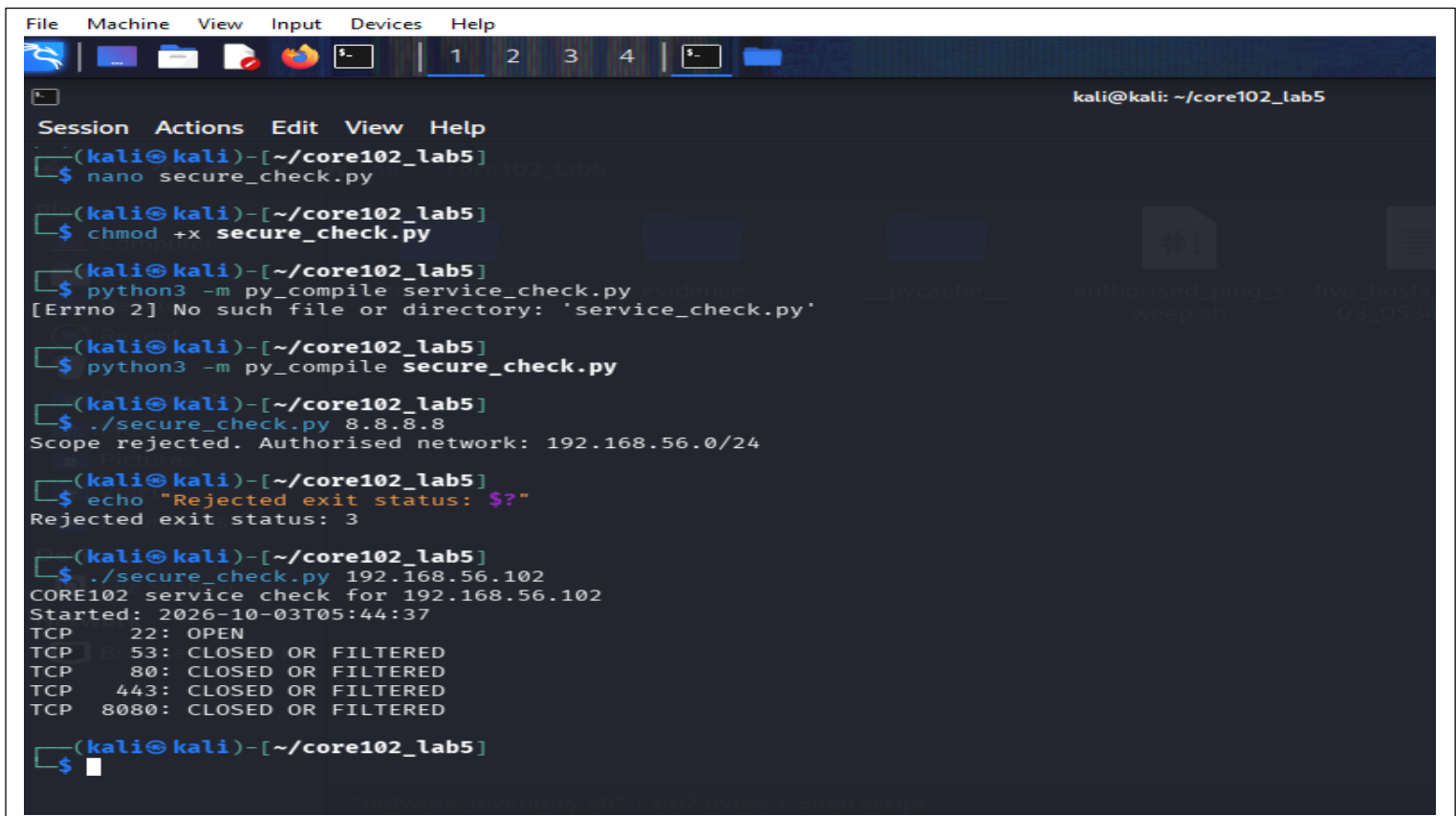
        raise SystemExit(2)

    if target not in AUTHORISED:
        print(f"Scope rejected. Authorised network: {AUTHORISED}")
        raise SystemExit(3)

    print(f"CORE102 service check for {target}")
    print(f"Started: {datetime.now().isoformat(timespec='seconds')}")
    for port in PORTS:
        try:
            with socket.create_connection((str(target), port), timeout=1):
                state = "OPEN"
        except (TimeoutError, ConnectionRefusedError, OSError):
            state = "CLOSED OR FILTERED"
        print(f"TCP {port:5}: {state}")

    chmod +x service_check.py
    python3 -m py_compile service_check.py
    ./service_check.py 8.8.8.8
    echo "Rejected exit status: $?"
    ./service_check.py 192.168.56.1

```



```

File  Machine  View  Input  Devices  Help
-----
kali@kali: ~/core102_lab5

Session  Actions  Edit  View  Help
(kali@kali)-[~/core102_lab5]
$ nano secure_check.py
(kali@kali)-[~/core102_lab5]
$ chmod +x secure_check.py
(kali@kali)-[~/core102_lab5]
$ python3 -m py_compile service_check.py
[Errno 2] No such file or directory: 'service_check.py'
(kali@kali)-[~/core102_lab5]
$ python3 -m py_compile secure_check.py
(kali@kali)-[~/core102_lab5]
$ ./secure_check.py 8.8.8.8
Scope rejected. Authorised network: 192.168.56.0/24
(kali@kali)-[~/core102_lab5]
$ echo "Rejected exit status: $?"
Rejected exit status: 3
(kali@kali)-[~/core102_lab5]
$ ./secure_check.py 192.168.56.102
CORE102 service check for 192.168.56.102
Started: 2026-10-03T05:44:37
TCP    22: OPEN
TCP    53: CLOSED OR FILTERED
TCP    80: CLOSED OR FILTERED
TCP    443: CLOSED OR FILTERED
TCP    8080: CLOSED OR FILTERED
(kali@kali)-[~/core102_lab5]
$

```

I tested it again after I have started the web server on kali and open the port on second server terminal .
when I run the script port 8080 is now open

```
ubuntu@ubuntu-server: ~/core102_lab4/web

Session Actions Edit View Help

(kali@kali)-[~/core102_lab5]
$ ssh ubuntu@192.168.56.102
Enter passphrase for key '/home/kali/.ssh/id_ed25519':
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-31-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Oct 3 10:09:02 AM UTC 2026

System load: 0.16           Memory usage: 15%      Processes:    106
Usage of /: 13.7% of 24.44GB Swap usage: 0%        Users logged in: 0

Expanded Security Maintenance for Applications is not enabled.

27 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Wed Sep 30 10:15:09 2026 from 192.168.56.101
ubuntu@ubuntu-server:~$ cd ~/core102_lab4/web && python3 -m http.server 8080 --
bind 0.0.0.0
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
^C
Keyboard interrupt received, exiting.
ubuntu@ubuntu-server:~/core102_lab4/web$

(kali@kali)-[~/core102_lab5]
$ ssh ubuntu@192.168.56.102
Enter passphrase for key '/home/kali/.s
Welcome to Ubuntu 26.04.1 LTS (GNU/Linu

 * Documentation:  https://docs.ubuntu.
 * Management:    https://landscape.ca
 * Support:       https://ubuntu.com/p

System information as of Sat Oct 3 10

System load: 0.16           Memor
Usage of /: 13.7% of 24.44GB Swap

(kali@kali)-[~/core102_lab
$

(kali@kali)-[~/core102_lab
$ ./secure_check.py 192.168.
CORE102 service check for 192.
2
Started: 2026-10-03T09:41:59
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: OPEN

(kali@kali)-[~/core102_lab
$
```

Part 5 Offline PCAP Summary with Scapy

I wrote `pcap_summary.py` using Scapy inside a Python virtual environment. It reads a saved capture and reports the packet total, protocol counts, the busiest endpoints and the most common TCP ports. I used `core102_http.pcap` from Lab 3 and saved the result in `pcap_summary.txt`. A missing file was rejected safely with exit status 2. I compared the packet total with `tcpdump -nn -r` and the numbers matched. I also added a DNS counter and tested it on `core102_dns.pcap`. A port number is only a convention, so this method can't prove which protocol was really used.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install scapy

#!/usr/bin/env python3
import collections
import sys
from pathlib import Path
from scapy.all import PcapReader, IP, TCP, UDP, ICMP

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} FILE.pcap")
    raise SystemExit(1)

pcap = Path(sys.argv[1])
if not pcap.is_file() or pcap.suffix.lower() not in ['.pcap', '.pcapng']:
```

```

print("Provide an existing .pcap or .pcapng file")
raise SystemExit(2)

protocols = collections.Counter()
endpoints = collections.Counter()
tcp_ports = collections.Counter()
packets = 0

with PcapReader(str(pcap)) as reader:
    for packet in reader:
        packets += 1
        if IP in packet:
            endpoints.update([packet[IP].src, packet[IP].dst])
            if TCP in packet:
                protocols['TCP'] += 1
                tcp_ports.update([packet[TCP].sport, packet[TCP].dport])
            elif UDP in packet:
                protocols['UDP'] += 1
            elif ICMP in packet:
                protocols['ICMP'] += 1
            else:
                protocols['Other IPv4'] += 1
        else:
            protocols['Non-IPv4'] += 1

print(f"File: {pcap}")
print(f"Packets: {packets}")
print("Protocols:")
for name, count in protocols.most_common():
    print(f"    {name:12} {count}")
print("Top endpoints:")
for address, count in endpoints.most_common(10):
    print(f"    {address:15} {count}")
print("Top TCP ports:")
for port, count in tcp_ports.most_common(10):
    print(f"    {port:5} {count}")

chmod +x pcap_summary.py
python3 -m py_compile pcap_summary.py
./pcap_summary.py missing.pcap
./pcap_summary.py core102_http.pcap | tee pcap_summary.txt

```

Session Actions Edit View Help

```
(kali@kali)-[~/core102_lab5]
$ python3 -m venv .venv
```

```
(kali@kali)-[~/core102_lab5]
$ source .venv/bin/activate
```

```
(.venv)-(kali@kali)-[~/core102_lab5]
```

```
$ python -m pip install scapy
```

Collecting scapy

Downloading scapy-2.8.0-py3-none-any.whl.metadata (5.8 kB)

Downloading scapy-2.8.0-py3-none-any.whl (2.8 MB)

2.8/2.8 MB 1.2 MB/s 0:00:02

Installing collected packages: scapy

Successfully installed scapy-2.8.0

```
(.venv)-(kali@kali)-[~/core102_lab5]
```

```
$ nano pcap_summary.py
```

```
(.venv)-(kali@kali)-[~/core102_lab5]
```

```
$ chmod +x pcap_summary.py
```

```
(.venv)-(kali@kali)-[~/core102_lab5]
```

```
$ python3 -m py_compile pcap_summary.py
```

```
(.venv)-(kali@kali)-[~/core102_lab5]
```

```
$
```

Working directory . I then added scapy source before installing python scapy .

I creat my python script paste my script and make it executable

```
(.venv)-(kali@kali)-[~/core102_lab5]
$ python3 -m py_compile pcap_summary.py

(.venv)-(kali@kali)-[~/core102_lab5]
$ ./pcap_summary.py missing.pcap
Provide an existing .pcap or .pcapng file

(.venv)-(kali@kali)-[~/core102_lab5]
$ ./pcap_summary.py core102_http.pcap | tee pcap_summary.txt
File: core102_http.pcap
Packets: 12
Protocols:
  TCP      12
Top endpoints:
  127.0.0.1      24
Top TCP ports:
  55884 12
  8080  12

(.venv)-(kali@kali)-[~/core102_lab5]
$
```

I first run my first script on a missing pcap file that does not exist , and it show an error message.

Before I run it on the actual pcap file and its show the file details .

Another pcap summary script on dns pcap

```
(.venv)-(kali@kali)-[~/core102_lab5]
$ ./pcap_summary.py core102_dns.pcap
File: core102_dns.pcap
Packets: 4
Protocols:
  UDP      4
Top endpoints:
  10.0.2.15      4
  192.168.1.1    4
Top TCP ports:
```

Part 6 Integrated Security Challenge

I confirmed the server is 192.168.56.102 and the client is 192.168.56.101 on the same private network. UFW is active with a default-deny incoming policy and allows SSH only from 192.168.56.0/24. I logged in from Kali with my SSH key and was asked for the key passphrase, not a password. I ran the inventory on the server and the sweep on the client, and the service checker showed only port 22 open. I captured one SSH connection and summarised it offline: it contained [N] TCP packets between the two lab machines, with port 22 at the top. I then stopped the test web server and the next check showed 8080 closed, which proved the checker detects the change.

Operational summary. The network is in its secured state: default-deny firewall, SSH allowed only from the lab subnet, and key-only login. Unresolved issues are that the DHCP service still listens on all interfaces, SSH is open to the whole lab subnet rather than only my client, and the sweep found 192.168.56.100, which [is the VirtualBox DHCP server / still needs to be identified]. For cleanup, I stopped the test web server and deleted the temporary 8080 firewall rule. I confirmed 8080 is closed and kept my private key out of the submission.

Control	Command or evidence	Result	Conclusion
Address and route			
Firewall policy			
SSH key authentication			
Listening services			
Host discovery			
PCAP summary			
Cleanup			

```
kali@kali: ~/core102_lab5
Session Actions Edit View Help
Usage of /: 13.7% of 24.44GB Users logged in: 0
Memory usage: 16% IPv4 address for enp0s3: 10.80.67.169
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.
27 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sat Oct 3 10:09:49 2026 from 192.168.56.101
ubuntu@ubuntu-server:~$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
enp0s3 UP 10.80.67.169/24 metric 100 fe80::a00:27ff:feb9:3d8
4/64
enp0s8 UP 192.168.56.102/24 metric 1024 fe80::a00:27ff:feb9:

(kali@kali)-[~/core102_lab5]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 10.80.67.204/24 fe80::fc2a:17d4:b059:edc7/64
eth1 UP 192.168.56.101/24 fe80::d9bd:df5b:c2a0:f2bd/64

(kali@kali)-[~/core102_lab5]
$ ip route
default via 10.80.67.147 dev eth0 proto dhcp src 10.80.67.204 metric 102
10.80.67.0/24 dev eth0 proto kernel scope link src 10.80.67.204 metric 102
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 101

(kali@kali)-[~/core102_lab5]
$
```

```
kali-linux-2026.2-virtualbox-amd64 (Snapshot 4) [Running] - Oracle VirtualBox : 1
File Machine View Input Devices Help

ubuntu@ubuntu-server: ~
Session Actions Edit View Help

This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.56.102' (ED25519) to the list of known hosts.
ubuntu@192.168.56.102: Permission denied (publickey).
ubuntu@ubuntu-server:~$ hostname
ubuntu-server
ubuntu@ubuntu-server:~$ ls
core102_lab2 core102_lab4 file.txt network_inventory.sh
ubuntu@ubuntu-server:~$ ./network_inventory.sh
Inventory saved to evidence/network_inventory_20261003_111053.txt
ubuntu@ubuntu-server:~$ sed -n '1,80p' evidence/network_inventory_*.txt
CORE102 NETWORK INVENTORY
Generated: 2026-10-03T11:10:53+00:00
Host: ubuntu-server
User: ubuntu

===== INTERFACES =====
lo UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
enp0s3 UP 08:00:27:b9:3d:84 <BROADCAST,MULTICAST,UP,LOWER_UP>
>
enp0s8 UP 08:00:27:b9:fd:97 <BROADCAST,MULTICAST,UP,LOWER_UP>
>
lo UNKNOWN 127.0.0.1/8 ::1/128
enp0s3 UP 10.80.67.169/24 metric 100 fe80::a00:27ff:feb9:3d8
4/64
enp0s8 UP 192.168.56.102/24 metric 1024 fe80::a00:27ff:feb9:
fd97/64

===== ROUTES =====
default via 10.80.67.147 dev enp0s3 proto dhcp src 10.80.67.169 metric 100
10.80.67.0/24 dev enp0s3 proto kernel scope link src 10.80.67.169 metric 100
10.80.67.147 dev enp0s3 proto dhcp scope link src 10.80.67.169 metric 100
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.102 metric 1024

(kali@kali)-[~/core102_lab5]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 10.80.67.204/24 fe80::fc2a:17d4:b059:edc7/64
eth1 UP 192.168.56.101/24 fe80::d9bd:df5b:c2a0:f2bd/64

(kali@kali)-[~/core102_lab5]
$ ip route
default via 10.80.67.147 dev eth0 proto dhcp src 10.80.67.204 metric 102
10.80.67.0/24 dev eth0 proto kernel scope link src 10.80.67.204 metric 102
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 101

(kali@kali)-[~/core102_lab5]
$ scp -i ~/.ssh/core102_ed25519 network_inventory.sh ubuntu@192.168.56.102:~/
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
network_inventory.sh 100% 667 347.5KB/s 00:00

(kali@kali)-[~/core102_lab5]
$
```

I runs on my Ubuntu server and it show me the file where I make it executable and run it .

```
kali@kali: ~/core102_lab5

Session Actions Edit View Help

#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .

===== DISK SPACE =====
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        25G   3.4G   20G   15% /
ubuntu@ubuntu-server:~$ grep -A12 "LISTENING SOCKETS" evidence/network_inventory_*.txt

===== LISTENING SOCKETS =====
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:67 0.0.0.0:*
udp UNCONN 0 0 192.168.56.102%enp0s8:68 0.0.0.0:*
udp UNCONN 0 0 10.80.67.169%enp0s3:68 0.0.0.0:*
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:*
udp UNCONN 0 0 [::1]:323 [::]:*
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
tcp LISTEN 0 4096 [::]:22 [::]:*

ubuntu@ubuntu-server:~$
```

```
(kali@kali)-[~/core102_lab5]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 10.80.67.204/24 fe80::fc2a:17d4:b059:ed
eth1 UP 192.168.56.101/24 fe80::d9bd:df5b:c2a0:

(kali@kali)-[~/core102_lab5]
$ ip route
default via 10.80.67.147 dev eth0 proto dhcp src 10.80.67.204 metric 10
10.80.67.0/24 dev eth0 proto kernel scope link src 10.80.67.204 metric
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 met

(kali@kali)-[~/core102_lab5]
$ scp -i ~/.ssh/core102_ed25519 network_inventory.sh ubuntu@192.168.5
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
network_inventory.sh 100% 667 347.5KB/s

(kali@kali)-[~/core102_lab5]
$ mkdir -p ~/core102_lab5/server_evidence

(kali@kali)-[~/core102_lab5]
$ scp -i ~/.ssh/core102_ed25519 'ubuntu@192.168.56.102:~/evidence/net
ory*.txt' ~/core102_lab5/server_evidence/
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
network_inventory_20261003_111053.txt 100% 3130 1.5MB/s

(kali@kali)-[~/core102_lab5]
$
```

```
(kali@kali)-[~/core102_lab5]
$ ip -br addr
lo UNKNOWN 127.0.0
eth0 UP 10.80.6
eth1 UP 192.168

(kali@kali)-[~/core102_lab5]
$ ip route
default via 10.80.67.147 dev eth0 proto
10.80.67.0/24 dev eth0 proto kernel sco
192.168.56.0/24 dev eth1 proto kernel s

(kali@kali)-[~/core102_lab5]
$ scp -i ~/.ssh/core102_ed25519 netwo
Enter passphrase for key '/home/kali/.s
network_inventory.sh

(kali@kali)-[~/core102_lab5]
$ mkdir -p ~/core102_lab5/server_evid

(kali@kali)-[~/core102_lab5]
$ scp -i ~/.ssh/core102_ed25519 'ubun
ory*.txt' ~/core102_lab5/server_eviden
Enter passphrase for key '/home/kali/.s
network_inventory_20261003_111053.txt

(kali@kali)-[~/core102_lab5]
$ ./authorised_ping_sweep.sh
Authorised ICMP sweep of 192.168.56.0/2
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded

(kali@kali)-[~/core102_lab5]
$ cat live_hosts*.txt
Authorised ICMP sweep of 192.168.56.0/2
4
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Authorised ICMP sweep of 192.168.56.0/2
4
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded

(kali@kali)-[~/core102_lab5]
$
```

```

(kali@kali)-[~/core102_lab5]
$ sudo ./authorised_ping_sweep.sh
[sudo] password for kali:
Authorised ICMP sweep of 192.168.56.0/24
Started: 2026-10-03T11:39:50-04:00
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Finished: 2026-10-03T11:44:02-04:00
Responsive hosts: 3
Saved: live_hosts_20261003_113950.txt

```

```

(kali@kali)-[~/core102_lab5]
$ ./secure_check.py 192.168.56.102
CORE102 service check for 192.168.56.102
Started: 2026-10-03T11:25:13
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

```

kali-linux-2026.2-virtualbox-amd64 (Snapshot 4) [Running] - Oracle VirtualBox : 1

File Machine View Input Devices Help

The screenshot displays a Kali Linux virtual machine interface with three terminal windows open:

- Left Terminal (ubuntu@ubuntu-server):** Shows the output of `ip -br addr` and `sudo ufw status`. The IP addresses listed are 127.0.0.1, 10.80.6, and 192.168.56.102. The ufw status shows it is active with logging on (low).
- Middle Terminal (kali@kali):** Shows the execution of `./authorised_ping_sweep.sh`. It reports an authorised ICMP sweep of 192.168.56.0/24, listing responsive hosts 192.168.56.100, 192.168.56.101, and 192.168.56.102.
- Right Terminal (kali@kali):** Shows the execution of `./secure_check.py 192.168.56.102`. It reports the CORE102 service check results for 192.168.56.102, showing TCP 22 as OPEN and other ports (53, 80, 443, 8080) as CLOSED OR FILTERED.

Part 7 Code Review Checklist

I checked that every script starts with its interpreter line, ran `bash -n` on the shell scripts and `py_compile` on the Python scripts, and searched them for passwords or keys. No syntax errors or secrets were found.

1. Which validation control most effectively prevented an out-of-scope action?

The scope check did, because it runs before anything is sent. When I ran the ping sweep against 203.0.113, the script rejected it and exited with status 2 without sending a single ping. The service checker did the same for 8.8.8.8 and exited with status 3. The approved port list was a useful second control, since it rejected port 3389. But the scope check mattered most, because it stops the action itself rather than just reporting on it afterwards.

2. What can a service checker conclude, and what can it not conclude, from a timeout?

When a port is open, the checker can confirm that something is listening and accepting connections. A timeout only tells me that the port did not answer within one second. It can't tell me why. The port could be closed, a firewall such as UFW could be dropping the packets, the host could be down, or the network could just be slow. That is why my script reports "CLOSED OR FILTERED" instead of claiming the port is closed. In my own tests, ports 80 and 443 timed out because nothing was listening and UFW drops unlisted traffic, but the script could not tell me which of those reasons applied.

3. How did automation improve repeatability compared with running commands manually?

Each script runs the same steps in the same order every time, with the same timeouts and the same output format, so the results can be compared between runs. The reports are timestamped and saved automatically, which gives me evidence I can show later. Typing commands by hand is easy to get wrong, because I can skip a step, mistype an address or forget to save the output. The scripts also check their input and stop safely on bad values, which I proved when they rejected invalid addresses and missing files. Running the service checker before and after stopping the test web server also gave me a clear, repeatable way to show a real change.

4. Which additional control would you add before these scripts could support a larger authorised environment?

I would add an approved target list kept in a separate file, together with logging of who ran each script, when, and against which targets. At the moment the scope is fixed in the code as 192.168.56.0/24, which works for this lab but is too rigid for a bigger environment. I would also add rate limiting so a scan can't overload the network, and a required authorisation check before the script runs. The logs would give an audit trail, and the target list would make sure every scan stays inside what has been approved.

